ScreenLink Project Report

Oct 10, 2026 · @Zain Mahedi

Summary

ScreenLink v1.1.0, released on 10 October 2026, lets an Android phone take a screenshot of a laptop on the same network, hand it to any AI app, and send the AI's answer back to be typed or clicked on the laptop. It has two parts: a Kotlin/Jetpack Compose Android app and a Python laptop client. The two talk over a small authenticated HTTP API on the local network, with no cloud backend.

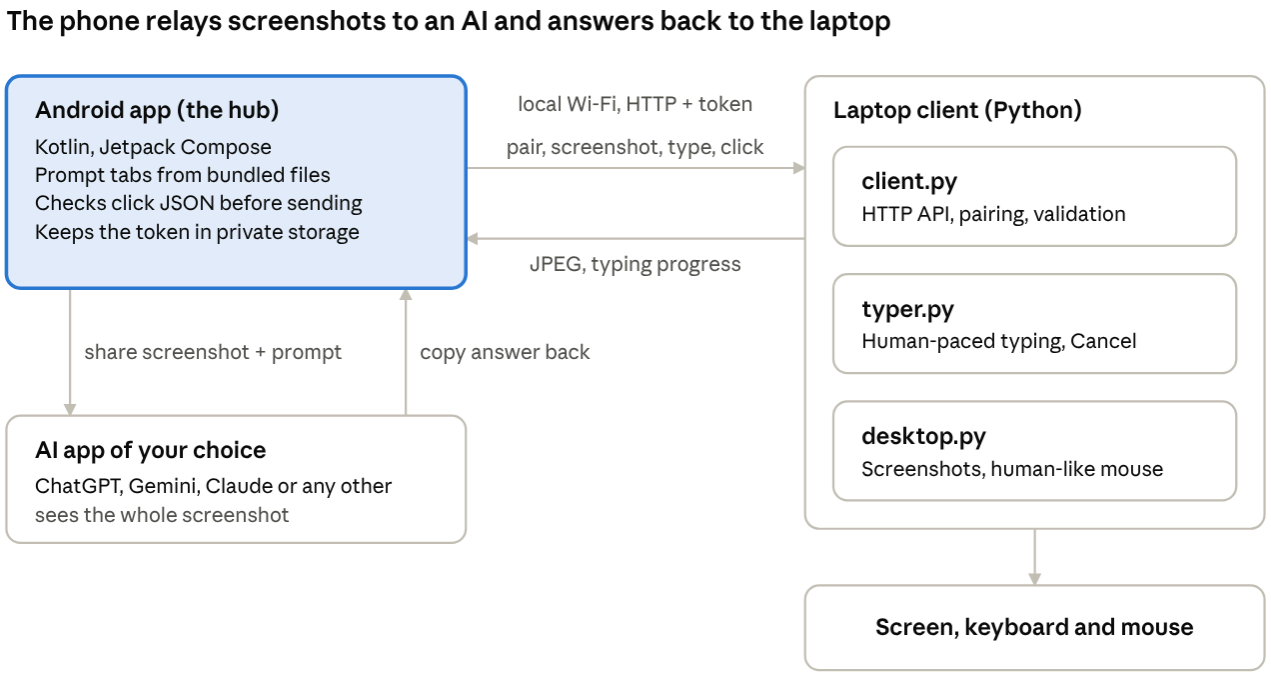
Item	Value
Current version	1.1.0 (versionCode 2), GitHub release
License	MIT, © 2026 MahediBuilds
Phone requirement	Android 8.0 or later (minSdk 26, targetSdk 36)
Laptop requirement	Windows 10/11 (primary) or macOS, Python 3.9+
Laptop dependencies	Pillow (screenshots), pynput (keyboard and mouse)
Size	About 3,200 lines of Kotlin, Python, Gradle and Markdown
Tests	54 laptop API tests (pytest) and Android unit tests for prompts and click parsing, all passing
Built-in prompts	Python and GUI (files in the app's prompts folder)

The 1.1.0 release changed two things. Clicks now move the mouse along a natural, human-speed path instead of jumping there. The bundled prompt set was cut down to Python and GUI.

The prompt files were not opened for this report. They are described only by their file names and by how the app loads them.

Architecture

The phone is the only part that talks to both sides. The laptop never contacts an AI service, and the AI app never contacts the laptop.



ScreenLink architecture · phone, AI app, laptop client

A typical round trip: capture a screenshot, share it with a prompt to the AI app, copy the answer back, then send it to the laptop to type or click. Nothing happens until the user presses a button on the phone.

Android app

The app is a single-screen Jetpack Compose app of about 1,200 lines of Kotlin, split into five source files. It holds no AI logic. It moves screenshots out to other apps through the share sheet and moves AI answers back to the laptop.

File	Lines	Responsibility
`MainActivity.kt`	478	Compose UI: Connection, Screenshot, Prompt and Response cards, zoom dialog, theme
`ScreenLinkViewModel.kt`	360	State, saved preferences, connect and pair, screenshot decoding, typing-job polling, clicks
`LaptopApi.kt`	182	Blocking HTTP client for the laptop API, with errors written for the user
`Prompts.kt`	109	`PromptLibrary` (loads prompt tabs from assets) and `ClickParser` (checks GUI replies)
`Sharing.kt`	88	Clipboard, share sheet, saving to Pictures/ScreenLink in the gallery

The screen is four cards, in the order you use them:

1. **Connection**: laptop IP and port, Connect, the pairing-code field, and Forget pairing.
2. **Screenshot**: capture, pinch-to-zoom preview, Share, and Save to gallery.
3. **Prompt**: one tab per bundled prompt file plus an editable Custom tab, with Copy, Share, and share-with-screenshot.
4. **Response**: paste the AI answer, then either **Send text to laptop** (with live progress and Cancel) or **Execute clicks on laptop**.

Key design points:

- **Prompts are files.** Every `.txt` in `assets/prompts/` becomes a tab. The order is General, Python and GUI when present, then the rest A to Z, then Custom. Empty, unreadable and `Custom.txt` files are skipped. Prompts are bundled at build time, so a prompt change needs a rebuild.
- **Click replies are checked on the phone first.** `ClickParser` pulls the JSON out of the reply even when it is wrapped in Markdown fences. It accepts only `click` actions with x and y between 0 and 1000, and rejects the whole reply if any step is invalid. The laptop then checks it again.
- **Errors are specific.** Timeouts, refused connections, no route, wrong service and not-paired each get their own message, so the user knows whether to check Wi-Fi, firewall, port or pairing.
- **Local data stays private.** The laptop address, pairing token, Custom prompt and last tab are kept in private preferences. `allowBackup` is off, and the shared screenshot is one cache file that is overwritten each time.

- **Permissions are minimal.** The app asks only for `INTERNET`, plus `WRITE_EXTERNAL_STORAGE` on Android 9 and older for saving to the gallery. Cleartext HTTP is allowed because the laptop serves plain HTTP on a private IP.

Laptop client

The laptop client is a dependency-light Python program, about 830 lines across four modules. It serves a seven-endpoint HTTP API built on the standard library's `ThreadingHTTPServer`. It runs only while you have it open in a console, and Ctrl+C stops it and cancels any typing in progress.

Module	Lines	Responsibility
`client.py`	451	HTTP server and routing, pairing and token checks, request validation, typing job, command line
`typer.py`	207	Types an answer line by line with human-like timing and a Cancel check on every key
`desktop.py`	128	DPI-aware screen size, JPEG screenshot, human-like mouse movement and click
`config.py`	42	Defaults, with optional overrides from `laptop/config.json` (unknown keys are rejected)

Method	Path	Auth	Purpose
GET	`/status`	none	Confirms it is ScreenLink, reports hostname and whether the caller's token is valid
POST	`/pair`	none	Trades the console pairing code for a token
GET	`/screenshot`	token	JPEG of the primary screen, with width and height headers
POST	`/type`	token	Starts a background typing job and returns 202
GET	`/type`	token	Progress of the current or last typing job
POST	`/type/cancel`	token	Cancels typing
POST	`/click`	token	Clicks up to 10 points given on a 0–1000 grid

How requests are handled:

- **One action at a time.** Typing and clicking share a lock, so a second action gets HTTP 409 instead of fighting over the keyboard or mouse.
- **Strict input checks.** Bodies must be JSON objects of 64 KB or less with a correct `Content-Type`. Text is capped at 20,000 characters and click requests at 10 steps.
- **Resolution-independent clicks.** Points arrive on a 0–1000 grid and are scaled to the real screen. Windows DPI awareness is switched on so that 125% or 150% display scaling does not move the click.
- **Port safety on Windows.** The socket claims the port exclusively, so two clients can't both bind it and split the phone's requests.
- **Tunable settings.** Port, JPEG quality, start delay, click limits and click interval all live in `config.py` and can be overridden per machine.

Human-like input

Both output paths, typing and clicking, deliberately imitate a person rather than a script. That keeps text editors happy and lets the user follow along. It also makes ScreenLink's input harder to tell apart from a real person's, which is why the Responsible use section below matters.

Typing (`typer.py`)

Behaviour	Setting
Speed	A random 10–50 words per minute, chosen again for every character
Punctuation	2.0–2.8× longer after `. ! ?`, 1.5–2.0× after `, ; :`, 1.2–1.7× around brackets and quotes
Word pauses	0.5–1.2 s after each word
Line pauses	0.3–0.9 s after each line, plus a 5 s pause every 4 lines
Typos	1.5% of letters get a wrong letter first, then Backspace and the right one
Start delay	1 s before the first key, so you can click into the target window
Long lines	Lines over 300 characters are skipped

Each line is typed without its leading indentation and ends with a space and Enter, which stops editors from accepting an autocomplete suggestion. `#IS` / `#IE` markers fix the indentation for editors that indent on their own: `#IE` presses Backspace once to leave a block. A one-line answer is typed with no trailing Enter, so it doesn't submit a form. Every key and every pause checks the Cancel flag, so Cancel on the phone stops typing within one character.

Mouse movement (`desktop.py`, new in 1.1.0)

Before 1.1.0 the pointer jumped straight to the target and clicked. It now glides there:

- **Duration follows Fitts's law.** 0.2 s + 0.09 s × $\log_2$(1 + distance/10), varied by ±15%. That is about 0.45 s for 50 px, 0.7 s for 500 px and 0.9 s across a 1920 px screen.
- **Curved path.** A cubic Bézier bows 3–12% of the distance to one side.
- **Natural speed curve.** A minimum-jerk profile accelerates out of the start and settles gently on the target, updating about 125 times a second. Timing is set by the clock, so slow `sleep` calls don't stretch the move.
- **Occasional overshoot.** About 35% of moves longer than 250 px go 4–14 px past the target and then correct.
- **Realistic click.** It pauses on the target for 80–200 ms, puts the pointer back on the exact point, then holds the button down for 50–110 ms.

All of these are named constants at the top of `desktop.py`, in the same style as the typing settings in `typer.py`.

Security and pairing

ScreenLink's access control is sound for a trusted home network. Its main weakness is that traffic is plain, unencrypted HTTP. Anyone who can read packets on the same Wi-Fi can see screenshots, typed text and the token.

Control	How it works
Pairing code	A 6-digit code from `secrets`, shown only in the laptop console, usable once, then replaced
Brute-force lock	5 wrong codes switch pairing off until the client restarts (HTTP 429)
Token	256-bit `secrets.token_urlsafe(32)`, sent as `Authorization: Bearer` on every control request

Control	How it works
Token storage	The laptop keeps only SHA-256 hashes in `paired.json` (git-ignored) and never logs tokens or typed text
Comparison	`hmac.compare_digest` against every paired device, so timing doesn't show which one matched
Revocation	`python client.py --reset-pairing` forgets every phone; Forget pairing on the phone drops its own token
Input limits	64 KB bodies, 20,000-character text, 10 click steps, coordinates 0–1000, JSON checked on both phone and laptop
Scope of action	Screenshot, type and click only. The client never runs code or shell commands from an AI answer
Human in the loop	Every action starts from a button the user presses on the phone; nothing runs on its own
Stalled connections	Dropped after 20 s; unexpected errors return 500 without crashing the server

Remaining exposure:

- **No encryption.** The client listens on `0.0.0.0:5001` over HTTP. On shared networks (university, office, café, hotel) a nearby attacker could capture the token and then take screenshots or type on the laptop.
- **Screenshots go to third parties.** The AI app gets the whole primary screen, including any email, passwords, chats or personal data visible at that moment.
- **The token never expires.** It stays valid until pairing is reset, so a lost or shared phone keeps access.
- **Clicks are trusted.** A wrong AI answer can click the wrong button. The phone shows the points first, but there is no on-laptop confirmation.

Vulnerabilities are reported privately as described in `SECURITY.md`.

Testing, build and release

Both test suites passed before the v1.1.0 release: 54 laptop tests and the Android unit tests. The new mouse-movement code has no automated test yet. It was checked by hand by generating paths without moving the real pointer.

Suite	Where	What it covers
Laptop API (54 tests, pytest)	`tests/test_client.py`	Runs the real HTTP server on 127.0.0.1 with a fake desktop and keyboard. Covers pairing and lockout, token checks, content type, size and JSON rejection, screenshots, typing (progress, cancel, errors, markers, typos, pauses), click scaling and validation, overlapping-action refusal, and exclusive port binding
Prompt loading (Kotlin, JUnit)	`PromptLibraryTest.kt`	Tab order, case handling, skipped files, labels, duplicate names, listing failure
Click parsing (Kotlin, JUnit)	`ClickParserTest.kt`	The GUI-prompt JSON format, Markdown fences and extra text, invalid replies

Building and shipping:

1. Run the tests: `python -m pytest tests` from the repo root, and `gradlew testDebugUnitTest` in `android/`.
2. Build: `gradlew assembleDebug` (about 10 MB) or `gradlew assembleRelease` (shrunk with R8, about 1.1 MB). A release build is signed only when `android/keystore.properties` exists; keys and APKs are git-ignored.
3. Raise `versionCode` and `versionName` in `android/app/build.gradle.kts`.
4. Install over USB: `adb install -r` with the matching build type.
5. Commit, tag (`vX.Y.Z`), push, and publish with `gh release create`, attaching the APK as `ScreenLink-vX.Y.Z.apk`.

Debug and release builds are signed with different keys and can't replace each other. The phone currently runs the 1.1.0 debug build. Moving it to the release APK means uninstalling first, which erases its pairing and settings.

The repository has 37 commits since the first one on 11 August 2026, with two tagged releases: v1.0.0 and v1.1.0. There is no CI; tests are run by hand before each release.

Responsible use

ScreenLink is a legitimate tool for remote control, accessibility and productivity on your own computer. But its mix of remote screen capture, AI answers, and typing and clicking that look human makes it easy to misuse. The project's README now carries a prominent

warning, and this section explains why.

The main risk is academic and assessment dishonesty. Screenshot to AI to human-paced typing is exactly the path someone would use to cheat on an online exam, a coding interview, a certification or a graded assignment. The human-like timing and mouse movement make that harder to spot. That is not their purpose, but it is a predictable effect.

Acceptable	Not acceptable
Controlling your own laptop from your phone	Exams, quizzes, interviews, certifications or graded work where outside or AI help is forbidden
Getting AI help where the rules allow it, such as personal projects and practice	Getting around proctoring, anti-cheat, bot detection or a platform's terms of service
Accessibility, such as entering text or clicking when using the laptop directly is hard	Using a computer you don't own or aren't authorised to control
Learning, demos and automating your own repetitive tasks	Capturing or sharing someone else's screen, messages or personal data without consent
	Impersonating a person, or automating fake activity, spam or fraud

Safe-use guidance:

- Run the client only on networks you trust, such as home Wi-Fi or your own hotspot, and never expose the port to the internet.
- Close or hide sensitive windows before each screenshot, because the whole screen goes to the AI app.
- Read the AI's answer before sending it, and watch the laptop while it types or clicks. Use Cancel if anything looks wrong.
- Stop the client (Ctrl+C) when you aren't using it, and run `--reset-pairing` if a phone is lost or the token may have leaked.
- Follow the rules of your school, employer and any platform you use. If you are unsure whether AI help is allowed, assume it isn't.

The user is responsible for how ScreenLink is used. The MIT license provides the software as is, without warranty.

Limitations and next steps

The biggest gaps are encryption, an on-screen sign that the laptop is being controlled, and automated checks. The steps below are ordered by impact.

Limitation	Effect	Suggested fix
Plain HTTP	Token, screenshots and text can be read on shared Wi-Fi	Self-signed TLS with the certificate fingerprint pinned at pairing time
No visible sign of remote control	People near the laptop can't tell it is being driven remotely	A tray icon or on-screen banner while typing or clicking
Tokens never expire	A lost phone keeps access until a reset	Token expiry, plus a list of paired devices that can be revoked one by one
Primary screen only	Screenshots and clicks ignore other monitors	Choose a monitor in the app
Click-only actions	GUI answers can't scroll, drag or press keys	New validated action types, each with its own test
Prompts bundled in the APK	Changing a prompt means a rebuild and reinstall	Optionally load prompts from the laptop or phone storage
No automated test of mouse paths	A regression in `move_path` would go unnoticed	Unit tests that the path ends on the target and timing stays in range, run without a real mouse
No CI	Tests depend on being run by hand	A GitHub Actions workflow for pytest and the Gradle unit tests on every push
Debug and release keys differ	Switching build types wipes the phone's pairing	Install only signed release builds on everyday devices

Recommended order:

- [ ] Add CI and a `move_path` unit test (small effort, protects every later change)
- [ ] Add a visible "being controlled" banner on the laptop (supports the responsible-use goals)
- [ ] Add token expiry and per-device revocation
- [ ] Add TLS with pinned certificates
- [ ] Add multi-monitor and new action types only if needed